

```

from google.colab import drive
drive.mount('/content/drive')

# Create a directory for this lab if it doesn't exist
import os
output_path = '/content/drive/MyDrive/CSET419_Lab11'
os.makedirs(output_path, exist_ok=True)
print(f'Outputs will be saved to: {output_path}')

Mounted at /content/drive
Outputs will be saved to: /content/drive/MyDrive/CSET419_Lab11

model_name = "gpt2"
tokenizer = GPT2Tokenizer.from_pretrained(model_name)
model = GPT2LMHeadModel.from_pretrained(model_name)

# GPT-2 does not have a padding token by default, so we add one
tokenizer.pad_token = tokenizer.eos_token
model.config.pad_token_id = model.config.eos_token_id

print(f"Model '{model_name}' loaded and tokenizer configured.")

/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your
settings tab (https://huggingface.co/settings/tokens), set it as
secret in your Google Colab and restart your session.
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to
access public models or datasets.
  warnings.warn(

{"model_id": "29d1ad8679db4749968d0787fda1b864", "version_major": 2, "version_minor": 0}

Warning: You are sending unauthenticated requests to the HF Hub.
Please set a HF_TOKEN to enable higher rate limits and faster
downloads.
WARNING:huggingface_hub.utils._http:Warning: You are sending
unauthenticated requests to the HF Hub. Please set a HF_TOKEN to
enable higher rate limits and faster downloads.

{"model_id": "effe4aefb8574db392eb3baacfc4059c", "version_major": 2, "version_minor": 0}

{"model_id": "6438dc855dc64567aa4189c9fe14665e", "version_major": 2, "version_minor": 0}

{"model_id": "554be62e6b714dbf931256c11b730d09", "version_major": 2, "version_minor": 0}

```

```
{"model_id": "c727f81e1a1a4dd3a24c2826a7693b10", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "1cb81e6da3014fe19325cde4323053fa", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "620fdad28fe54a109718d35b2a545b44", "version_major": 2, "version_minor": 0}
```

GPT2LMHeadModel LOAD REPORT from: gpt2

Key	Status		
h.{0...11}.attn.bias	UNEXPECTED		

Notes:

- UNEXPECTED : can be ignored when loading from different task/architecture; not ok if you expect identical arch.

```
{"model_id": "9e91fff64cfb4070a474e2494e14f3c9", "version_major": 2, "version_minor": 0}
```

Model 'gpt2' loaded and tokenizer configured.

```
from datasets import Dataset
```

```
# Sample data for Product Review Generator as per typical lab requirements
```

```
reviews_data = [  
    {"text": "Product: Smartphone | Review: This phone has an amazing camera and long battery life. Highly recommend!"},  
    {"text": "Product: Laptop | Review: Great performance for gaming, but the fans can get a bit loud under load."},  
    {"text": "Product: Headphones | Review: Excellent sound quality and very comfortable for long listening sessions."},  
    {"text": "Product: Smartwatch | Review: The fitness tracking is accurate, but the screen is a bit small for my liking."}  
]
```

```
def tokenize_function(examples):  
    return tokenizer(examples["text"], truncation=True,  
padding="max_length", max_length=128)
```

```
# Create dataset
```

```
review_dataset = Dataset.from_list(reviews_data)  
tokenized_reviews = review_dataset.map(tokenize_function,  
batched=True)
```

```
print("Product review dataset prepared and tokenized.")
```

```
{"model_id": "ba6bdb9ceb684d6fb695bdad32d054f6", "version_major": 2, "version_minor": 0}
```

Product review dataset prepared and tokenized.

```
!pip install transformers datasets torch
import torch
from transformers import GPT2Tokenizer, GPT2LMHeadModel, Trainer,
TrainingArguments

print("Libraries installed. GPU available:",
torch.cuda.is_available())

Requirement already satisfied: transformers in
/usr/local/lib/python3.12/dist-packages (5.0.0)
Requirement already satisfied: datasets in
/usr/local/lib/python3.12/dist-packages (4.0.0)
Requirement already satisfied: torch in
/usr/local/lib/python3.12/dist-packages (2.10.0+cu128)
Requirement already satisfied: filelock in
/usr/local/lib/python3.12/dist-packages (from transformers) (3.25.2)
Requirement already satisfied: huggingface-hub<2.0,>=1.3.0 in
/usr/local/lib/python3.12/dist-packages (from transformers) (1.8.0)
Requirement already satisfied: numpy>=1.17 in
/usr/local/lib/python3.12/dist-packages (from transformers) (2.0.2)
Requirement already satisfied: packaging>=20.0 in
/usr/local/lib/python3.12/dist-packages (from transformers) (26.0)
Requirement already satisfied: pyyaml>=5.1 in
/usr/local/lib/python3.12/dist-packages (from transformers) (6.0.3)
Requirement already satisfied: regex!=2019.12.17 in
/usr/local/lib/python3.12/dist-packages (from transformers)
(2025.11.3)
Requirement already satisfied: tokenizers<=0.23.0,>=0.22.0 in
/usr/local/lib/python3.12/dist-packages (from transformers) (0.22.2)
Requirement already satisfied: typer-slim in
/usr/local/lib/python3.12/dist-packages (from transformers) (0.24.0)
Requirement already satisfied: safetensors>=0.4.3 in
/usr/local/lib/python3.12/dist-packages (from transformers) (0.7.0)
Requirement already satisfied: tqdm>=4.27 in
/usr/local/lib/python3.12/dist-packages (from transformers) (4.67.3)
Requirement already satisfied: pyarrow>=15.0.0 in
/usr/local/lib/python3.12/dist-packages (from datasets) (18.1.0)
Requirement already satisfied: dill<0.3.9,>=0.3.0 in
/usr/local/lib/python3.12/dist-packages (from datasets) (0.3.8)
Requirement already satisfied: pandas in
/usr/local/lib/python3.12/dist-packages (from datasets) (2.2.2)
Requirement already satisfied: requests>=2.32.2 in
/usr/local/lib/python3.12/dist-packages (from datasets) (2.32.4)
Requirement already satisfied: xxhash in
/usr/local/lib/python3.12/dist-packages (from datasets) (3.6.0)
Requirement already satisfied: multiprocessing<0.70.17 in
/usr/local/lib/python3.12/dist-packages (from datasets) (0.70.16)
Requirement already satisfied: fsspec<=2025.3.0,>=2023.1.0 in
```

```
/usr/local/lib/python3.12/dist-packages (from
fsspec[http]<=2025.3.0,>=2023.1.0->datasets) (2025.3.0)
Requirement already satisfied: typing-extensions>=4.10.0 in
/usr/local/lib/python3.12/dist-packages (from torch) (4.15.0)
Requirement already satisfied: setuptools in
/usr/local/lib/python3.12/dist-packages (from torch) (75.2.0)
Requirement already satisfied: sympy>=1.13.3 in
/usr/local/lib/python3.12/dist-packages (from torch) (1.14.0)
Requirement already satisfied: networkx>=2.5.1 in
/usr/local/lib/python3.12/dist-packages (from torch) (3.6.1)
Requirement already satisfied: jinja2 in
/usr/local/lib/python3.12/dist-packages (from torch) (3.1.6)
Requirement already satisfied: cuda-bindings==12.9.4 in
/usr/local/lib/python3.12/dist-packages (from torch) (12.9.4)
Requirement already satisfied: nvidia-cuda-nvrtc-cu12==12.8.93 in
/usr/local/lib/python3.12/dist-packages (from torch) (12.8.93)
Requirement already satisfied: nvidia-cuda-runtime-cu12==12.8.90 in
/usr/local/lib/python3.12/dist-packages (from torch) (12.8.90)
Requirement already satisfied: nvidia-cuda-cupti-cu12==12.8.90 in
/usr/local/lib/python3.12/dist-packages (from torch) (12.8.90)
Requirement already satisfied: nvidia-cudnn-cu12==9.10.2.21 in
/usr/local/lib/python3.12/dist-packages (from torch) (9.10.2.21)
Requirement already satisfied: nvidia-cublas-cu12==12.8.4.1 in
/usr/local/lib/python3.12/dist-packages (from torch) (12.8.4.1)
Requirement already satisfied: nvidia-cufft-cu12==11.3.3.83 in
/usr/local/lib/python3.12/dist-packages (from torch) (11.3.3.83)
Requirement already satisfied: nvidia-curand-cu12==10.3.9.90 in
/usr/local/lib/python3.12/dist-packages (from torch) (10.3.9.90)
Requirement already satisfied: nvidia-cusolver-cu12==11.7.3.90 in
/usr/local/lib/python3.12/dist-packages (from torch) (11.7.3.90)
Requirement already satisfied: nvidia-cusparselt-cu12==0.7.1 in
/usr/local/lib/python3.12/dist-packages (from torch) (0.7.1)
Requirement already satisfied: nvidia-nccl-cu12==2.27.5 in
/usr/local/lib/python3.12/dist-packages (from torch) (2.27.5)
Requirement already satisfied: nvidia-nvshmem-cu12==3.4.5 in
/usr/local/lib/python3.12/dist-packages (from torch) (3.4.5)
Requirement already satisfied: nvidia-nvtx-cu12==12.8.90 in
/usr/local/lib/python3.12/dist-packages (from torch) (12.8.90)
Requirement already satisfied: nvidia-nvjitlink-cu12==12.8.93 in
/usr/local/lib/python3.12/dist-packages (from torch) (12.8.93)
Requirement already satisfied: nvidia-cufile-cu12==1.13.1.3 in
/usr/local/lib/python3.12/dist-packages (from torch) (1.13.1.3)
Requirement already satisfied: triton==3.6.0 in
/usr/local/lib/python3.12/dist-packages (from torch) (3.6.0)
Requirement already satisfied: cuda-pathfinder~=1.1 in
/usr/local/lib/python3.12/dist-packages (from cuda-bindings==12.9.4-
>torch) (1.5.0)
```

Requirement already satisfied: aiohttp!=4.0.0a0,! =4.0.0a1 in /usr/local/lib/python3.12/dist-packages (from fsspec[http]<=2025.3.0,>=2023.1.0->datasets) (3.13.4)

Requirement already satisfied: hf-xet<2.0.0,>=1.4.2 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<2.0,>=1.3.0->transformers) (1.4.2)

Requirement already satisfied: httpx<1,>=0.23.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<2.0,>=1.3.0->transformers) (0.28.1)

Requirement already satisfied: typer in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<2.0,>=1.3.0->transformers) (0.24.1)

Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests>=2.32.2->datasets) (3.4.6)

Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests>=2.32.2->datasets) (3.11)

Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests>=2.32.2->datasets) (2.5.0)

Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests>=2.32.2->datasets) (2026.2.25)

Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from sympy>=1.13.3->torch) (1.3.0)

Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from jinja2->torch) (3.0.3)

Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas->datasets) (2.9.0.post0)

Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas->datasets) (2025.2)

Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas->datasets) (2025.3)

Requirement already satisfied: aiohappyeyeballs>=2.5.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,! =4.0.0a1->fsspec[http]<=2025.3.0,>=2023.1.0->datasets) (2.6.1)

Requirement already satisfied: aiosignal>=1.4.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,! =4.0.0a1->fsspec[http]<=2025.3.0,>=2023.1.0->datasets) (1.4.0)

Requirement already satisfied: attrs>=17.3.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,! =4.0.0a1->fsspec[http]<=2025.3.0,>=2023.1.0->datasets) (26.1.0)

Requirement already satisfied: frozenlist>=1.1.1 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,! =4.0.0a1->fsspec[http]<=2025.3.0,>=2023.1.0->datasets) (1.4.0)

```

=4.0.0a1->fsspec[http]<=2025.3.0,>=2023.1.0->datasets) (1.8.0)
Requirement already satisfied: multidict<7.0,>=4.5 in
/usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!
=4.0.0a1->fsspec[http]<=2025.3.0,>=2023.1.0->datasets) (6.7.1)
Requirement already satisfied: propcache>=0.2.0 in
/usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!
=4.0.0a1->fsspec[http]<=2025.3.0,>=2023.1.0->datasets) (0.4.1)
Requirement already satisfied: yarl<2.0,>=1.17.0 in
/usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!
=4.0.0a1->fsspec[http]<=2025.3.0,>=2023.1.0->datasets) (1.23.0)
Requirement already satisfied: anyio in
/usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0-
>huggingface-hub<2.0,>=1.3.0->transformers) (4.13.0)
Requirement already satisfied: httpcore==1.* in
/usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0-
>huggingface-hub<2.0,>=1.3.0->transformers) (1.0.9)
Requirement already satisfied: h11>=0.16 in
/usr/local/lib/python3.12/dist-packages (from httpcore==1.*-
>httpx<1,>=0.23.0->huggingface-hub<2.0,>=1.3.0->transformers) (0.16.0)
Requirement already satisfied: six>=1.5 in
/usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2-
>pandas->datasets) (1.17.0)
Requirement already satisfied: click>=8.2.1 in
/usr/local/lib/python3.12/dist-packages (from typer->huggingface-
hub<2.0,>=1.3.0->transformers) (8.3.1)
Requirement already satisfied: shellingham>=1.3.0 in
/usr/local/lib/python3.12/dist-packages (from typer->huggingface-
hub<2.0,>=1.3.0->transformers) (1.5.4)
Requirement already satisfied: rich>=12.3.0 in
/usr/local/lib/python3.12/dist-packages (from typer->huggingface-
hub<2.0,>=1.3.0->transformers) (13.9.4)
Requirement already satisfied: annotated-doc>=0.0.2 in
/usr/local/lib/python3.12/dist-packages (from typer->huggingface-
hub<2.0,>=1.3.0->transformers) (0.0.4)
Requirement already satisfied: markdown-it-py>=2.2.0 in
/usr/local/lib/python3.12/dist-packages (from rich>=12.3.0->typer-
>huggingface-hub<2.0,>=1.3.0->transformers) (4.0.0)
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in
/usr/local/lib/python3.12/dist-packages (from rich>=12.3.0->typer-
>huggingface-hub<2.0,>=1.3.0->transformers) (2.20.0)
Requirement already satisfied: mdurl~=0.1 in
/usr/local/lib/python3.12/dist-packages (from markdown-it-py>=2.2.0-
>rich>=12.3.0->typer->huggingface-hub<2.0,>=1.3.0->transformers)
(0.1.2)
Libraries installed. GPU available: True

```

```
!pip install transformers datasets accelerate -q
```

```

import torch, math
from transformers import GPT2LMHeadModel, GPT2Tokenizer, Trainer,

```

```

    TrainingArguments, DataCollatorForLanguageModeling, set_seed)
from datasets import Dataset
import os

set_seed(42)

model_name = 'gpt2'
tokenizer = GPT2Tokenizer.from_pretrained(model_name)
model = GPT2LMHeadModel.from_pretrained(model_name)

tokenizer.pad_token = tokenizer.eos_token
model.config.pad_token_id = tokenizer.eos_token_id

print("Model and tokenizer re-initialized for Component-I.")

{"model_id": "366d5e725a6e45528d450d73b7dc4482", "version_major": 2, "version_minor": 0}

GPT2LMHeadModel LOAD REPORT from: gpt2
Key | Status | |
-----+-----+---+
h.{0...11}.attn.bias | UNEXPECTED | |

Notes:
- UNEXPECTED : can be ignored when loading from different
task/architecture; not ok if you expect identical arch.

Model and tokenizer re-initialized for Component-I.

def generate_text(model, tokenizer, prompt, max_length=100):
    model.eval()
    inputs = tokenizer.encode(prompt,
return_tensors='pt').to(model.device)
    with torch.no_grad():
        out = model.generate(inputs, max_length=max_length,
temperature=0.8,
top_k=50, do_sample=True,
pad_token_id=tokenizer.eos_token_id)
    return tokenizer.decode(out[0], skip_special_tokens=True)

review_prompts = [
    'This product is',
    'I bought this phone and',
    'The quality of this item',
]

print('=== BASELINE REVIEWS (Before Fine-Tuning) ===')
baseline = {}
for p in review_prompts:
    baseline[p] = generate_text(model, tokenizer, p)
    print(f'Prompt: {p}\nOutput: {baseline[p]}\n')

```

The attention mask is not set and cannot be inferred from input because pad token is same as eos token. As a consequence, you may observe unexpected behavior. Please pass your input's `attention\_mask` to obtain reliable results.

=== BASELINE REVIEWS (Before Fine-Tuning) ===

Prompt: This product is

Output: This product is made from high quality, lightweight stainless steel. If you are looking for something a little more durable, it's a good choice.

Laser Pouch

Not all of our laser printers are created equal. We have a laser printer that comes with all of our printer parts. These parts include our new 3D printer and a 3D printed printing service. All of our printers make laser printers, including our laser printers, using laser technology. Our laser printers are the most

Prompt: I bought this phone and

Output: I bought this phone and I have not used it on a lot of people. I have also not used it on any other people.

The screen was amazing and the sound was amazing. It was not loud. I would never use it on a tv, laptop, smartphone or other connected device in the future.

The battery life is good. The phone works great but it has so many problems.

I have been using phones that have the Snapdragon 616 processor with the 8

Prompt: The quality of this item

Output: The quality of this item in the item description (and if the item is already in stock) will determine how many times the customer calls you to confirm availability.

This item is currently out of stock and may be out of stock based on the availability of your specific request.

Please contact us before placing your order to verify availability.

Your order is now up for pre-order.

Step 1: Setup, Load Model and Drive

```
from google.colab import drive
import torch, math, os
from transformers import (GPT2LMHeadModel, GPT2Tokenizer, Trainer,
                          TrainingArguments, DataCollatorForLanguageModeling, set_seed)
```



```

from datasets import Dataset

drive.mount('/content/drive')
output_path = '/content/drive/MyDrive/CSET419_Lab11'
os.makedirs(output_path, exist_ok=True)

set_seed(42)
model_name = 'gpt2'
tokenizer = GPT2Tokenizer.from_pretrained(model_name)
model = GPT2LMHeadModel.from_pretrained(model_name)

tokenizer.pad_token = tokenizer.eos_token
model.config.pad_token_id = tokenizer.eos_token_id

print(f"Setup complete. Outputs will be saved to: {output_path}")

Drive already mounted at /content/drive; to attempt to forcibly
remount, call drive.mount("/content/drive", force_remount=True).

{"model_id": "e3921dabe8584d0da90ea5f020fa2f6e", "version_major": 2, "vers
ion_minor": 0}

GPT2LMHeadModel LOAD REPORT from: gpt2
Key | Status | |
-----+-----+---+
h.{0...11}.attn.bias | UNEXPECTED | |

Notes:
- UNEXPECTED :can be ignored when loading from different
task/architecture; not ok if you expect identical arch.

Setup complete. Outputs will be saved to:
/content/drive/MyDrive/CSET419_Lab11

# Component-I: Step 2 - Baseline Reviews
def generate_text(model, tokenizer, prompt, max_length=100):
    model.eval()
    inputs = tokenizer.encode(prompt,
return_tensors='pt').to(model.device)
    with torch.no_grad():
        out = model.generate(inputs, max_length=max_length,
temperature=0.8,
        top_k=50, do_sample=True,
pad_token_id=tokenizer.eos_token_id)
    return tokenizer.decode(out[0], skip_special_tokens=True)

review_prompts = ['This product is', 'I bought this phone and', 'The
quality of this item']

print('=== BASELINE REVIEWS (Before Fine-Tuning) ===')
baseline = {p: generate_text(model, tokenizer, p) for p in

```

```
review_prompts}
for p, output in baseline.items():
    print(f'Prompt: {p}\nOutput: {output}\n')
```

=== BASELINE REVIEWS (Before Fine-Tuning) ===

Prompt: This product is

Output: This product is made from high quality, lightweight stainless steel. If you are looking for something a little more durable, it's a good choice.

Laser Pouch

Not all of our laser printers are created equal. We have a laser printer that comes with all of our printer parts. These parts include our new 3D printer and a 3D printed printing service. All of our printers make laser printers, including our laser printers, using laser technology. Our laser printers are the most

Prompt: I bought this phone and

Output: I bought this phone and I have not used it on a lot of people. I have also not used it on any other people.

The screen was amazing and the sound was amazing. It was not loud. I would never use it on a tv, laptop, smartphone or other connected device in the future.

The battery life is good. The phone works great but it has so many problems.

I have been using phones that have the Snapdragon 616 processor with the 8

Prompt: The quality of this item

Output: The quality of this item in the item description (and if the item is already in stock) will determine how many times the customer calls you to confirm availability.

This item is currently out of stock and may be out of stock based on the availability of your specific request.

Please contact us before placing your order to verify availability.

Your order is now up for pre-order.

Component-I: Step 3 - Prepare Dataset and Fine-Tune

```
corpus = [
```

```
    'this phone has an amazing battery life and the camera quality is outstanding for the price.',
```

```
    'i bought this laptop for college and it handles all my assignments and coding projects perfectly.',
```

```
'the sound quality of these headphones is incredible with deep
bass and clear vocals.',
'this smartwatch tracks my steps accurately and the heart rate
monitor is very reliable.',
'great wireless earbuds with noise cancellation that blocks out
all background sound.',
'the keyboard feels very comfortable for long typing sessions and
the backlight is a nice touch.',
'this portable charger saved me during travel and it charges my
phone three times on a single charge.',
'the tablet screen is bright and colorful which makes watching
movies a great experience.',
'i love this fitness tracker because it motivates me to reach my
daily exercise goals.',
'this bluetooth speaker is compact but delivers surprisingly loud
and clear audio.',
'the delivery was fast and the product was packed securely with no
damage at all.',
'excellent value for money and the build quality feels premium
despite the affordable price.',
'the customer service team was very helpful when i had questions
about the product features.',
'this camera takes stunning photos in low light and the video
recording quality is very smooth.',
'i have been using this product for three months and it still
works perfectly like day one.',
'the design is sleek and modern and it looks great on my desk next
to my other gadgets.',
'easy to set up right out of the box and the instructions were
clear and simple to follow.',
'highly recommend this product to anyone looking for quality and
reliability at a fair price.',
'the software updates keep adding new features which makes this
purchase even more worthwhile.',
'best purchase i made this year and i would definitely buy from
this brand again.',
]
```

```
dataset = Dataset.from_dict({'text': corpus})
tokenized = dataset.map(lambda x: tokenizer(x['text'],
truncation=True, max_length=128, padding='max_length'), batched=True)
tokenized = tokenized.map(lambda x: {'labels': x['input_ids']},
batched=True)
split = tokenized.train_test_split(test_size=0.15, seed=42)

data_collator = DataCollatorForLanguageModeling(tokenizer=tokenizer,
mlm=False)
training_args = TrainingArguments(
    output_dir='./gpt2-reviews', num_train_epochs=15,
```

```

per_device_train_batch_size=4,
    learning_rate=5e-5, weight_decay=0.01, warmup_steps=50,
eval_strategy='epoch',
    logging_steps=10, save_strategy='no',
fp16=torch.cuda.is_available(), report_to='none'
)

trainer = Trainer(model=model, args=training_args,
train_dataset=split['train'], eval_dataset=split['test'],
data_collator=data_collator)
trainer.train()

# Save Component 1
model.save_pretrained(os.path.join(output_path,
'component1_product_reviews'))

{"model_id": "45547bb5dd3441da8f844591aece9e29", "version_major": 2, "version_minor": 0}

{"model_id": "79044a415bc5420583be717341ff690b", "version_major": 2, "version_minor": 0}

<IPython.core.display.HTML object>

{"model_id": "ec1f3cae142947acbeca0d5e10e5e661", "version_major": 2, "version_minor": 0}

# Component-I: Step 4 - Generate and Compare
eval_res = trainer.evaluate()
print(f'Perplexity: {math.exp(eval_res["eval_loss"]):.2f}')

print('\n=== FINE-TUNED REVIEWS (After Fine-Tuning) ===')
for p in review_prompts:
    ft_out = generate_text(model, tokenizer, p)
    print(f'Prompt: {p}\n Baseline: {baseline[p][:100]}...\n Fine-Tuned: {ft_out[:100]}\n')

<IPython.core.display.HTML object>

Perplexity: 24.74

=== FINE-TUNED REVIEWS (After Fine-Tuning) ===
Prompt: This product is
    Baseline: This product is made from high quality, lightweight stainless steel. If you are looking for somethin...
    Fine-Tuned: This product is packed with features that make this purchase even more worthwhile. The quality of th

Prompt: I bought this phone and
    Baseline: I bought this phone and I have not used it on a lot of people. I have also not used it on any other ...

```

Fine-Tuned: I bought this phone and it handles all my daily chores perfectly. I would definitely buy from this b

Prompt: The quality of this item

Baseline: The quality of this item in the item description (and if the item is already in stock) will determin...

Fine-Tuned: The quality of this item is exemplary with very little distortion and no audible hum. The build qual

Component-II: Recipe Instruction Generator

```
recipes_corpus = [
    {"text": "Dish: Scrambled Eggs | Instructions: Whisk eggs in a bowl, add salt and pepper. Heat butter in a pan and cook until soft and fluffy."},
    {"text": "Dish: Pasta Marinara | Instructions: Boil water and cook pasta. In another pan, heat olive oil and garlic, add tomato sauce, then mix with pasta."},
    {"text": "Dish: Grilled Cheese | Instructions: Butter two slices of bread. Place cheese between them and grill on a skillet until golden brown on both sides."},
    {"text": "Dish: Fruit Salad | Instructions: Chop apples, bananas, and grapes. Toss them in a bowl with a splash of lime juice and honey."},
    {"text": "Dish: Guacamole | Instructions: Mash ripe avocados in a bowl. Stir in chopped onions, cilantro, lime juice, and a pinch of salt."}
]
```

```
recipe_ds = Dataset.from_list(recipes_corpus).map(lambda x:
tokenizer(x['text'], truncation=True, padding='max_length',
max_length=128), batched=True)
recipe_ds = recipe_ds.map(lambda x: {'labels': x['input_ids']},
batched=True)
```

```
recipe_args = TrainingArguments(
    output_dir='./gpt2-recipes', num_train_epochs=10,
    per_device_train_batch_size=2,
    learning_rate=5e-5, save_strategy='no', report_to='none',
    fp16=torch.cuda.is_available()
)
```

```
recipe_trainer = Trainer(model=model, args=recipe_args,
train_dataset=recipe_ds, data_collator=data_collator)
recipe_trainer.train()
```

```
model.save_pretrained(os.path.join(output_path,
'component2_recipe_generator'))
print('\n=== TESTING RECIPE GENERATOR ===')
```

```

print(generate_text(model, tokenizer, "Dish: Vegetable Stir-fry |
Instructions:", max_length=60))

{"model_id": "e7590bc1f36f4e0fb8048ca6412bf94b", "version_major": 2, "version_minor": 0}

{"model_id": "7e4503a6a6fc46fc88eda5d411c2eac4", "version_major": 2, "version_minor": 0}

<IPython.core.display.HTML object>

{"model_id": "b3085dda26074b48870ec06b8124d1de", "version_major": 2, "version_minor": 0}

```

=== TESTING RECIPE GENERATOR ===

Dish: Vegetable Stir-fry | Instructions: Heat oil in pan and add onion, garlic, and lime juice. Cook until smooth and tender, about 5 minutes on each side. Place on a baking sheet lined with parchment paper. Drizzle with olive oil and sprinkle with salt and pepper

Component-II: Recipe Instruction Generator

1. Prepare Recipe Dataset

```

recipes_corpus = [
    {"text": "Dish: Scrambled Eggs | Instructions: Whisk eggs in a bowl, add salt and pepper. Heat butter in a pan and cook until soft and fluffy."},
    {"text": "Dish: Pasta Marinara | Instructions: Boil water and cook pasta. In another pan, heat olive oil and garlic, add tomato sauce, then mix with pasta."},
    {"text": "Dish: Grilled Cheese | Instructions: Butter two slices of bread. Place cheese between them and grill on a skillet until golden brown on both sides."},
    {"text": "Dish: Fruit Salad | Instructions: Chop apples, bananas, and grapes. Toss them in a bowl with a splash of lime juice and honey."},
    {"text": "Dish: Guacamole | Instructions: Mash ripe avocados in a bowl. Stir in chopped onions, cilantro, lime juice, and a pinch of salt."}
]

```

```

recipe_dataset = Dataset.from_list(recipes_corpus)
tokenized_recipes = recipe_dataset.map(lambda x: tokenizer(x['text'],
truncation=True,
padding='max_length', max_length=128), batched=True)
tokenized_recipes = tokenized_recipes.map(lambda x: {'labels':
x['input_ids']}, batched=True)

```

2. Fine-tune for Recipes

```

recipe_training_args = TrainingArguments(

```

```

    output_dir='./gpt2-recipes',
    num_train_epochs=10,
    per_device_train_batch_size=2,
    learning_rate=5e-5,
    save_strategy='no',
    report_to='none',
    fp16=torch.cuda.is_available()
)

recipe_trainer = Trainer(
    model=model, # Continuing from the current state or re-loading
base gpt2
    args=recipe_training_args,
    train_dataset=tokenized_recipes,
    data_collator=data_collator
)

print("Starting fine-tuning for Recipe Generator...")
recipe_trainer.train()

# 3. Save Recipe Model
recipe_save_path =
'/content/drive/MyDrive/CSET419_Lab11/component2_recipe_generator'
model.save_pretrained(recipe_save_path)
tokenizer.save_pretrained(recipe_save_path)
print(f"Recipe model saved to {recipe_save_path}")

# 4. Final Verification
print("\n=== TESTING RECIPE GENERATOR ===")
recipe_prompt = "Dish: Vegetable Stir-fry | Instructions:"
print(generate_text(model, tokenizer, recipe_prompt, max_length=60))

{"model_id": "5701c726a4f049f0ab29670b3ecb2dd2", "version_major": 2, "version_minor": 0}

{"model_id": "1fb410300bd442c1b8f48da31c73ab76", "version_major": 2, "version_minor": 0}

Starting fine-tuning for Recipe Generator...
<IPython.core.display.HTML object>

{"model_id": "b7e29c7f993a45ef82d80557976777be", "version_major": 2, "version_minor": 0}

Recipe model saved to
/content/drive/MyDrive/CSET419_Lab11/component2_recipe_generator

=== TESTING RECIPE GENERATOR ===
Dish: Vegetable Stir-fry | Instructions: Heat oil in pan and add
onion, garlic, and lime juice. Cook until smooth and tender, about 5

```

minutes on each side. Place on a baking sheet lined with parchment paper. Drizzle with olive oil and sprinkle with salt and pepper

Component-II: Recipe Instruction Generator

1. Prepare Recipe Dataset

```
recipes_corpus = [
    {"text": "Dish: Scrambled Eggs | Instructions: Whisk eggs in a bowl, add salt and pepper. Heat butter in a pan and cook until soft and fluffy."},
    {"text": "Dish: Pasta Marinara | Instructions: Boil water and cook pasta. In another pan, heat olive oil and garlic, add tomato sauce, then mix with pasta."},
    {"text": "Dish: Grilled Cheese | Instructions: Butter two slices of bread. Place cheese between them and grill on a skillet until golden brown on both sides."},
    {"text": "Dish: Fruit Salad | Instructions: Chop apples, bananas, and grapes. Toss them in a bowl with a splash of lime juice and honey."},
    {"text": "Dish: Guacamole | Instructions: Mash ripe avocados in a bowl. Stir in chopped onions, cilantro, lime juice, and a pinch of salt."}
]
```

```
from datasets import Dataset
recipe_dataset = Dataset.from_list(recipes_corpus)
tokenized_recipes = recipe_dataset.map(lambda x: tokenizer(x['text'],
truncation=True,
padding='max_length', max_length=128), batched=True)
tokenized_recipes = tokenized_recipes.map(lambda x: {'labels':
x['input_ids']}, batched=True)
```

2. Fine-tune for Recipes

```
recipe_training_args = TrainingArguments(
    output_dir='./gpt2-recipes',
    num_train_epochs=10,
    per_device_train_batch_size=2,
    learning_rate=5e-5,
    save_strategy='no',
    report_to='none',
    fp16=torch.cuda.is_available()
)
```

```
recipe_trainer = Trainer(
    model=model,
    args=recipe_training_args,
    train_dataset=tokenized_recipes,
    data_collator=data_collator
)
```



```

print("Starting fine-tuning for Recipe Generator...")
recipe_trainer.train()

# 3. Save Recipe Model to Drive
recipe_save_path =
'/content/drive/MyDrive/CSET419_Lab11/component2_recipe_generator'
model.save_pretrained(recipe_save_path)
tokenizer.save_pretrained(recipe_save_path)
print(f"\nRecipe model saved to {recipe_save_path}")

# 4. Final Verification
print("\n=== TESTING RECIPE GENERATOR ===")
recipe_prompt = "Dish: Vegetable Stir-fry | Instructions:"
# Using the generate_text function defined in Step 2
print(generate_text(model, tokenizer, recipe_prompt, max_length=60))

{"model_id": "d1a2b09c45f74c219ed1dc15d2c0f94d", "version_major": 2, "version_minor": 0}

{"model_id": "c8b72747012944fe905dd0a75722435e", "version_major": 2, "version_minor": 0}

Starting fine-tuning for Recipe Generator...

<IPython.core.display.HTML object>

{"model_id": "4d9ce9c667c3474290aed9289195758d", "version_major": 2, "version_minor": 0}

Recipe model saved to
/content/drive/MyDrive/CSET419_Lab11/component2_recipe_generator

=== TESTING RECIPE GENERATOR ===
Dish: Vegetable Stir-fry | Instructions: Butter two slices of avocados
in a bowl. Stir in chopped onions, cilantro, lime juice, and a pinch
of salt. Toss in chopped tomatoes, lime juice, and a pinch of salt.
Heat olive oil in a

# Component-II: Recipe Instruction Generator

# 1. Prepare Recipe Dataset
recipes_corpus = [
    {"text": "Dish: Scrambled Eggs | Instructions: Whisk eggs in a
bowl, add salt and pepper. Heat butter in a pan and cook until soft
and fluffy."},
    {"text": "Dish: Pasta Marinara | Instructions: Boil water and cook
pasta. In another pan, heat olive oil and garlic, add tomato sauce,
then mix with pasta."},
    {"text": "Dish: Grilled Cheese | Instructions: Butter two slices
of bread. Place cheese between them and grill on a skillet until

```

```

golden brown on both sides."},
    {"text": "Dish: Fruit Salad | Instructions: Chop apples, bananas,
and grapes. Toss them in a bowl with a splash of lime juice and
honey."},
    {"text": "Dish: Guacamole | Instructions: Mash ripe avocados in a
bowl. Stir in chopped onions, cilantro, lime juice, and a pinch of
salt."}
]

recipe_dataset = Dataset.from_list(recipes_corpus)
tokenized_recipes = recipe_dataset.map(lambda x: tokenizer(x['text'],
truncation=True,
padding='max_length', max_length=128), batched=True)
tokenized_recipes = tokenized_recipes.map(lambda x: {'labels':
x['input_ids']}, batched=True)

# 2. Fine-tune for Recipes
recipe_training_args = TrainingArguments(
    output_dir='./gpt2-recipes',
    num_train_epochs=10,
    per_device_train_batch_size=2,
    learning_rate=5e-5,
    save_strategy='no',
    report_to='none',
    fp16=torch.cuda.is_available()
)

recipe_trainer = Trainer(
    model=model,
    args=recipe_training_args,
    train_dataset=tokenized_recipes,
    data_collator=data_collator
)

print("Starting fine-tuning for Recipe Generator...")
recipe_trainer.train()

# 3. Save Recipe Model to Drive
recipe_save_path =
'/content/drive/MyDrive/CSET419_Lab11/component2_recipe_generator'
model.save_pretrained(recipe_save_path)
tokenizer.save_pretrained(recipe_save_path)
print(f"\nRecipe model saved to {recipe_save_path}")

# 4. Final Verification
print("\n=== TESTING RECIPE GENERATOR ===")
recipe_prompt = "Dish: Vegetable Stir-fry | Instructions:"
# Using the generate_text function defined in the previous step
print(generate_text(model, tokenizer, recipe_prompt, max_length=60))

```

```
{"model_id":"411b1491065649909eca4d9b46fc725b","version_major":2,"version_minor":0}
```

```
{"model_id":"37bd4cec255f46c18f52e71ec3bc5e7c","version_major":2,"version_minor":0}
```

Starting fine-tuning for Recipe Generator...

<IPython.core.display.HTML object>

```
{"model_id":"1034702fb11d4675bc0a33c0fabd6e4f","version_major":2,"version_minor":0}
```

Recipe model saved to
/content/drive/MyDrive/CSET419_Lab11/component2_recipe_generator

=== TESTING RECIPE GENERATOR ===

Dish: Vegetable Stir-fry | Instructions: Butter two slices of bread. Place cheese between them and grill on a skillet until golden brown on both sides. Place tomato sauce in a bowl and mix with onions, garlic, chilli, lime juice, and a pinch of salt. Stir

```
corpus = [  
    'this phone has an amazing battery life and the camera quality is  
    outstanding for the price.',  
    'i bought this laptop for college and it handles all my  
    assignments and coding projects perfectly.',  
    'the sound quality of these headphones is incredible with deep  
    bass and clear vocals.',  
    'this smartwatch tracks my steps accurately and the heart rate  
    monitor is very reliable.',  
    'great wireless earbuds with noise cancellation that blocks out  
    all background sound.',  
    'the keyboard feels very comfortable for long typing sessions and  
    the backlight is a nice touch.',  
    'this portable charger saved me during travel and it charges my  
    phone three times on a single charge.',  
    'the tablet screen is bright and colorful which makes watching  
    movies a great experience.',  
    'i love this fitness tracker because it motivates me to reach my  
    daily exercise goals.',  
    'this bluetooth speaker is compact but delivers surprisingly loud  
    and clear audio.',  
    'the delivery was fast and the product was packed securely with no  
    damage at all.',  
    'excellent value for money and the build quality feels premium  
    despite the affordable price.',  
    'the customer service team was very helpful when i had questions  
    about the product features.',  
    'this camera takes stunning photos in low light and the video
```

```
    'recording quality is very smooth.',
    'i have been using this product for three months and it still
    works perfectly like day one.',
    'the design is sleek and modern and it looks great on my desk next
    to my other gadgets.',
    'easy to set up right out of the box and the instructions were
    clear and simple to follow.',
    'highly recommend this product to anyone looking for quality and
    reliability at a fair price.',
    'the software updates keep adding new features which makes this
    purchase even more worthwhile.',
    'best purchase i made this year and i would definitely buy from
    this brand again.',
]
```

```
dataset = Dataset.from_dict({'text': corpus})
tokenized = dataset.map(lambda x: tokenizer(x['text']),
truncation=True,
    max_length=128, padding='max_length'), batched=True,
remove_columns=['text'])
```

```
# Ensure labels are added for loss calculation
```

```
tokenized = tokenized.map(lambda x: {'labels': x['input_ids']},
batched=True)
```

```
split = tokenized.train_test_split(test_size=0.15, seed=42)
```

```
data_collator = DataCollatorForLanguageModeling(tokenizer=tokenizer,
mlm=False)
```

```
training_args = TrainingArguments(
    output_dir='./gpt2-reviews',
    num_train_epochs=15,
    per_device_train_batch_size=4,
    learning_rate=5e-5,
    weight_decay=0.01,
    warmup_steps=50,
    eval_strategy='epoch',
    logging_steps=10,
    save_strategy='no',
    fp16=torch.cuda.is_available(),
    report_to='none'
)
```

```
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=split['train'],
    eval_dataset=split['test'],
    data_collator=data_collator
```

```

)

trainer.train()

{"model_id": "d173346ed69d4e43a7895ae18cc0c006", "version_major": 2, "version_minor": 0}

{"model_id": "83c09105220d48abbab5897e0324610c", "version_major": 2, "version_minor": 0}

<IPython.core.display.HTML object>

TrainOutput(global_step=75, training_loss=1.9250287294387818,
metrics={'train_runtime': 9.8613, 'train_samples_per_second': 25.859,
'train_steps_per_second': 7.605, 'total_flos': 16657367040000.0,
'train_loss': 1.9250287294387818, 'epoch': 15.0})

eval_res = trainer.evaluate()
print(f'Perplexity: {math.exp(eval_res["eval_loss"]):.2f}')

print('\n=== FINE-TUNED REVIEWS (After Fine-Tuning) ===')
for p in review_prompts:
    ft_out = generate_text(model, tokenizer, p)
    print(f'Prompt: {p}')
    print(f' Baseline: {baseline[p][:120]}...')
    print(f' Fine-Tuned: {ft_out[:120]}\n')

# Save to Drive as previously requested
final_save_path =
'/content/drive/MyDrive/CSET419_Lab11/component1_product_reviews'
model.save_pretrained(final_save_path)
tokenizer.save_pretrained(final_save_path)
print(f'Model saved to {final_save_path}')

<IPython.core.display.HTML object>

Perplexity: 24.74

=== FINE-TUNED REVIEWS (After Fine-Tuning) ===
Prompt: This product is
    Baseline: This product is made from high quality, lightweight
stainless steel. If you are looking for something a little more
dura...
    Fine-Tuned: This product is packed with features that make this
purchase even more worthwhile. The quality of this product exceeds e

Prompt: I bought this phone and
    Baseline: I bought this phone and I have not used it on a lot of
people. I have also not used it on any other people.

The screen ...

```

Fine-Tuned: I bought this phone and it handles all my daily chores perfectly. I would definitely buy from this brand again.

Verifie

Prompt: The quality of this item

Baseline: The quality of this item in the item description (and if the item is already in stock) will determine how many times the...

Fine-Tuned: The quality of this item is exemplary with very little distortion and no audible hum. The build quality is outstanding f

```
{"model_id":"7e6d5569cc734963b4e9375cad8a0d2e","version_major":2,"version_minor":0}
```

Model saved to

/content/drive/MyDrive/CSET419_Lab11/component1_product_reviews

Sample data for Recipe Instruction Generator

```
recipes_data = [  
    {"text": "Dish: Scrambled Eggs | Instructions: Whisk eggs in a  
bowl, add salt and pepper. Heat butter in a pan and cook until soft  
and fluffy."},  
    {"text": "Dish: Pasta Marinara | Instructions: Boil water and cook  
pasta. In another pan, heat olive oil and garlic, add tomato sauce,  
then mix with pasta."},  
    {"text": "Dish: Grilled Cheese | Instructions: Butter two slices  
of bread. Place cheese between them and grill on a skillet until  
golden brown on both sides."}  
]
```

Create recipe dataset

```
recipe_dataset = Dataset.from_list(recipes_data)  
tokenized_recipes = recipe_dataset.map(tokenize_function,  
batched=True)
```

Training setup for Recipe Generator

```
recipe_output_path = os.path.join(output_path,  
'fine_tuned_gpt2_recipes')  
training_args_recipe = TrainingArguments(  
    output_dir=os.path.join(output_path, 'results_recipe'),  
    num_train_epochs=5,  
    per_device_train_batch_size=1,  
    save_total_limit=1,  
    report_to='none'  
)
```

```
trainer_recipe = Trainer(  
    model=model,  
    args=training_args_recipe,  
    train_dataset=tokenized_recipes,
```

```

    data_collator=data_collator,
)

print('Starting fine-tuning for Recipe Generator...')
trainer_recipe.train()

model.save_pretrained(recipe_output_path)
tokenizer.save_pretrained(recipe_output_path)
print(f'Recipe model saved to {recipe_output_path}')

{"model_id":"7942ea96db5a4bfc8d248d42ac266f39","version_major":2,"version_minor":0}

Starting fine-tuning for Recipe Generator...
<IPython.core.display.HTML object>

{"model_id":"11d87b54d0f94f1a903b514f378952e6","version_major":2,"version_minor":0}

{"model_id":"7c6b8fe9c80342699abae6dae64c34ef","version_major":2,"version_minor":0}

Recipe model saved to
/content/drive/MyDrive/CSET419_Lab11/fine_tuned_gpt2_recipes

import torch
from transformers import pipeline

# Load the fine-tuned model for inference
fine_tuned_model_path = os.path.join(output_path,
'fine_tuned_gpt2_reviews')

if os.path.exists(fine_tuned_model_path):
    review_gen = pipeline('text-generation',
model=fine_tuned_model_path, tokenizer=fine_tuned_model_path)

    prompt = "Product: Wireless Mouse | Review:"
    generated = review_gen(prompt, max_length=50,
num_return_sequences=1)

    print("\n--- Generated Review ---")
    print(generated[0]['generated_text'])
else:
    print("Waiting for training to finish and model to be saved...")

{"model_id":"b5472f3d95b34c03ba505a1a61d5ea50","version_major":2,"version_minor":0}

Passing `generation_config` together with generation-related
arguments=({'max_length', 'num_return_sequences'}) is deprecated and
will be removed in future versions. Please pass either a

```

`generation\_config` object OR all generation parameters explicitly, but not both.
Both `max\_new\_tokens` (=256) and `max\_length` (=50) seem to have been set. `max\_new\_tokens` will take precedence. Please refer to the documentation for more information.
(https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)

--- Generated Review ---

Product: Wireless Mouse | Review: Great for gaming, great sound, great battery life, and solid sound. I have two other mice and they sound great.

Verified purchase: This mouse is good for gaming.

Verified purchase: This mouse is great for gaming.

Verified purchase: This mouse is good for gaming.

Verified purchase: This mouse is good for gaming.

Verified purchase: This mouse is good for gaming.

Verified purchase: This mouse is good for gaming.

Verified purchase: Great sound for gaming.

Verified purchase: Great sound for gaming.

Verified purchase: Great sound for gaming.

Verified purchase: Great sound for gaming.

Verified purchase: Great sound for gaming.

Verified purchase: Great sound for gaming.

Verified purchase: Great sound for gaming.

Verified purchase: Great sound for gaming.

Verified purchase: Great sound for gaming.

Verified purchase: Great sound for gaming.

Verified purchase: Great sound for gaming.

Verified purchase: Great sound for gaming.

Verified purchase: Great sound for gaming.

Verified purchase: Great sound for gaming

```
from transformers import DataCollatorForLanguageModeling

# Prepare data collator for causal LM
tokenizer.pad_token = tokenizer.eos_token
data_collator = DataCollatorForLanguageModeling(tokenizer=tokenizer,
mlm=False)

training_args = TrainingArguments(
    output_dir=os.path.join(output_path, 'results'),
    overwrite_output_dir=True,
    num_train_epochs=3,
    per_device_train_batch_size=2,
    save_steps=10,
    save_total_limit=2,
    logging_dir=os.path.join(output_path, 'logs'),
    report_to='none'
)

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=tokenized_reviews,
    data_collator=data_collator,
)

print('Training setup complete. Starting fine-tuning...')
trainer.train()

# Save the fine-tuned model to Drive
model.save_pretrained(os.path.join(output_path,
'fine_tuned_gpt2_reviews'))
tokenizer.save_pretrained(os.path.join(output_path,
'fine_tuned_gpt2_reviews'))
print(f'Model saved to {output_path}/fine_tuned_gpt2_reviews')
```

```
-----
-----
TypeError                                Traceback (most recent call
last)
/tmp/ipykernel_3273/3487514390.py in <cell line: 0>()
      5 data_collator =
DataCollatorForLanguageModeling(tokenizer=tokenizer, mlm=False)
      6
----> 7 training_args = TrainingArguments(
      8     output_dir=os.path.join(output_path, 'results'),
      9     overwrite_output_dir=True,
```

```
TypeError: TrainingArguments.__init__() got an unexpected keyword argument 'overwrite_output_dir'
```

```
from transformers import DataCollatorForLanguageModeling,
TrainingArguments, Trainer
import os

# Prepare data collator for causal LM
tokenizer.pad_token = tokenizer.eos_token
data_collator = DataCollatorForLanguageModeling(tokenizer=tokenizer,
mlm=False)
```

```
training_args = TrainingArguments(
    output_dir=os.path.join(output_path, 'results'),
    num_train_epochs=3,
    per_device_train_batch_size=1,
    save_steps=5,
    save_total_limit=1,
    logging_steps=1,
    logging_dir=os.path.join(output_path, 'logs'),
    report_to='none'
)
```

```
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=tokenized_reviews,
    data_collator=data_collator,
)
```

```
print('Starting fine-tuning...')
trainer.train()
```

```
# Save the fine-tuned model to Drive
final_path = os.path.join(output_path, 'fine_tuned_gpt2_reviews')
model.save_pretrained(final_path)
tokenizer.save_pretrained(final_path)
print(f'Model and tokenizer saved to {final_path}')
```

```
`logging_dir` is deprecated and will be removed in v5.2. Please set
`TENSORBOARD_LOGGING_DIR` instead.
```

```
Starting fine-tuning...
```

```
`loss_type=None` was set in the config but it is unrecognized. Using
the default loss: `ForCausalLMLoss`.
```

```
<IPython.core.display.HTML object>
```

```
{"model_id": "938a718542694ae896f145e1c86fcda0", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "f1d6be9ef95c452586f0186fde56a119", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "3f1cef44b7e3462b9c99d01af2ad2fbf", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "49a25d26b25f4c28bcfce01144f2d177", "version_major": 2, "version_minor": 0}
```

Model and tokenizer saved to
/content/drive/MyDrive/CSET419_Lab11/fine_tuned_gpt2_reviews

```
!pip install python-docx
```

Collecting python-docx

Downloading python_docx-1.2.0-py3-none-any.whl.metadata (2.0 kB)

Requirement already satisfied: lxml>=3.1.0 in

/usr/local/lib/python3.12/dist-packages (from python-docx) (6.0.2)

Requirement already satisfied: typing_extensions>=4.9.0 in

/usr/local/lib/python3.12/dist-packages (from python-docx) (4.15.0)

Downloading python_docx-1.2.0-py3-none-any.whl (252 kB)

```
0.0/253.0 kB ? eta -:-:-
253.0/253.0 kB 9.7 MB/s eta
```

0:00:00

```
import os
from docx import Document
```

```
# Check files in /content to help user debug
print('Files in /content:', os.listdir('/content'))
```

```
# Path to the assignment file
doc_path = '/content/CSET419_Lab11_FineTuning.docx'
```

```
if os.path.exists(doc_path):
    doc = Document(doc_path)
    # Extracting text to see the instructions
    instructions = [p.text for p in doc.paragraphs if p.text.strip() != '']
    print('\nFile found and loaded. Extracting instructions:')
    for line in instructions[:10]:
        print(line)
else:
    print(f'\nError: File {doc_path} not found. Please upload it to the Files sidebar (left pane) and try again.')
```

```
Files in /content: ['.config', 'CSET419_Lab11_FineTuning.docx', 'drive', 'sample_data']
```

```
File found and loaded. Extracting instructions:
CSET419 – Introduction to Generative AI
```

Lab – 11

Objective

The objective of this lab is to fine-tune a pre-trained generative model (GPT-2) for real-world applications. Students will fine-tune GPT-2 to build a Product Review Generator for e-commerce and a Recipe Instruction Generator for a food-tech application, learning how transfer learning adapts a general model to specific business domains.

Learning Outcomes

After completing this lab, students will be able to:

- Understand how fine-tuning applies to real-world industry applications

- Load and configure a pre-trained GPT-2 model using Hugging Face

- Transformers

- Prepare real-world domain-specific datasets for causal language modeling

- Fine-tune the model and compare generated output before and after training